```sql
CREATE TABLE user_activity (
    user_id    INT,
    event_type VARCHAR(50),
    amount     INT,
    event_time TIMESTAMP
);
```

Output

SQL query successfully executed. However, the result set is empty.

```sql
SELECT * FROM user_activity;
```

Output

user_id	event_type	amount	event_time
1	login	0	2024-01-01 10:00:00
1	purchase	200	2024-01-01 11:00:00
1	login	0	2024-01-02 09:00:00
2	login	0	2024-01-01 10:30:00
2	purchase	500	2024-01-03 12:00:00
3	login	0	2024-01-02 14:00:00

```sql
SELECT COUNT(*) AS total_rows FROM user_activity;
```

Output

total_rows
6

```sql
  #WINDOW FUNCTION
SELECT user_id, COUNT(*) AS total_events
FROM user_activity
GROUP BY user_id
ORDER BY user_id;
```

Output

user_id	total_events
1	3
2	2
3	1

```sql
#RANKING FUNCTIONS
SELECT
    user_id,
    amount,
    event_time,
    RANK()       OVER (PARTITION BY user_id ORDER BY amount DESC) AS rank_val,
    DENSE_RANK() OVER (PARTITION BY user_id ORDER BY amount DESC) AS dense_rank_val,
    ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY amount DESC) AS row_num
FROM user_activity
WHERE event_type = 'purchase';
```

Output

user_id	amount	event_time	rank_val	dense_rank_val	row_num
1	200	2024-01-01 11:00:00	1	1	1
2	500	2024-01-03 12:00:00	1	1	1

```sql
#LATEST ACTIVITY PER USER
SELECT user_id, event_type, amount, event_time
FROM (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_time DESC) AS rn
    FROM user_activity
) ranked
WHERE rn = 1;
```

Output

user_id	event_type	amount	event_time
1	login	0	2024-01-02 09:00:00
2	purchase	500	2024-01-03 12:00:00
3	login	0	2024-01-02 14:00:00

```sql
# LAG AND LEAD
SELECT
    user_id,
    amount,
    event_time,
    LAG(amount)       OVER (PARTITION BY user_id ORDER BY event_time) AS prev_amount,
    LEAD(event_time)  OVER (PARTITION BY user_id ORDER BY event_time) AS
next_event_time
FROM user_activity
WHERE event_type = 'purchase';
```

Output

user_id	amount	event_time	prev_amount	next_event_time
1	200	2024-01-01 11:00:00		
2	500	2024-01-03 12:00:00		

```sql
#CTE Latest Purchase Per User
WITH purchase_cte AS (
    SELECT * FROM user_activity
    WHERE event_type = 'purchase'
),
ranked_purchases AS (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_time DESC) AS rn
    FROM purchase_cte
)
SELECT user_id, amount, event_time
FROM ranked_purchases
WHERE rn = 1;
```

Output

user_id	amount	event_time
1	200	2024-01-01 11:00:00
2	500	2024-01-03 12:00:00

main.py

Share Run

```python
# PART 3 (already done)
# PART 3: FUNCTIONAL PROGRAMMING

# Task 1: Data
data = [
    {"user_id": 1, "event_type": "purchase", "amount": 200},
    {"user_id": 2, "event_type": "purchase", "amount": 500},
    {"user_id": 3, "event_type": "login",    "amount": 0}
]
print("Task 1 - Data:", data)

# Task 2: filter_purchases
def filter_purchases(data):
    return list(filter(lambda x: x["event_type"] == "purchase", data))

purchases = filter_purchases(data)
print("Task 2 - Filtered:", purchases)

# Task 3: map
amounts = list(map(lambda x: x["amount"], purchases))
print("Task 3 - Amounts:", amounts)

# Task 4: lambda discount
apply_discount = lambda amount: round(amount * 0.90, 2)
discounted = list(map(apply_discount, amounts))
print("Task 4 - Discounted:", discounted)

# Task 5: Higher-Order Function
def apply_function_to_list(func, lst):
    return list(map(func, lst))

result = apply_function_to_list(apply_discount, amounts)
print("Task 5 - Higher-Order Result:", result)


# PART 4: MODULES & PACKAGES

# processor.py logic
def filter_purchases_processor(data):
    return [record for record in data if record["event_type"] == "purchase"]

# utils/helper.py logic
def convert_amounts_to_float(amounts_list):
    return [float(a) for a in amounts_list]

# main.py logic
purchase_records = filter_purchases_processor(data)
raw_amounts = [r["amount"] for r in purchase_records]
float_amounts = convert_amounts_to_float(raw_amounts)

print("Part 4 - Filtered:", purchase_records)
print("Part 4 - Floats:", float_amounts)
```

```python

def safe_filter_purchases(data):
    try:
        if not isinstance(data, list):
            raise TypeError(f"Expected a list, got {type(data).__name__}")
        if len(data) == 0:
            raise ValueError("Data list is empty. Nothing to process.")
        for i, record in enumerate(data):
            if not isinstance(record, dict):
                raise TypeError(f"Record {i} is not a dict")
            if "event_type" not in record or "amount" not in record:
                raise KeyError(f"Record {i} missing required keys")
        result = [r for r in data if r["event_type"] == "purchase"]

    except (TypeError, ValueError, KeyError) as e:
        print(f"  [ERROR] {type(e).__name__}: {e}")
        return []
    else:
        print(f"  [SUCCESS] Found {len(result)} purchase record(s).")
        return result
    finally:
        print("  [FINALLY] Execution complete.")

print("\nTest 1 - Valid Data:")
safe_filter_purchases(data)

print("Test 2 - Empty List:")
safe_filter_purchases([])

print("Test 3 - Invalid Type:")
safe_filter_purchases("bad input")

print("Test 4 - Missing Keys:")
safe_filter_purchases([{"user_id": 99}])
```

```
Task 1 - Data: [{'user_id': 1, 'event_type': 'purchase', 'amount': 200},
    {'user_id': 2, 'event_type': 'purchase', 'amount': 500}, {'user_id': 3,
    'event_type': 'login', 'amount': 0}]
Task 2 - Filtered: [{'user_id': 1, 'event_type': 'purchase', 'amount': 200},
    {'user_id': 2, 'event_type': 'purchase', 'amount': 500}]
Task 3 - Amounts: [200, 500]
Task 4 - Discounted: [180.0, 450.0]
Task 5 - Higher-Order Result: [180.0, 450.0]
Part 4 - Filtered: [{'user_id': 1, 'event_type': 'purchase', 'amount': 200},
    {'user_id': 2, 'event_type': 'purchase', 'amount': 500}]
Part 4 - Floats: [200.0, 500.0]

Test 1 - Valid Data:
ERROR!
  [SUCCESS] Found 2 purchase record(s).
  [FINALLY] Execution complete.
Test 2 - Empty List:
  [ERROR] ValueError: Data list is empty. Nothing to process.
  [FINALLY] Execution complete.
Test 3 - Invalid Type:
  [ERROR] TypeError: Expected a list, got str
  [FINALLY] Execution complete.
Test 4 - Missing Keys:
  [ERROR] KeyError: 'Record 0 missing required keys'
  [FINALLY] Execution complete.

=== Code Execution Successful ===
```